

Technical Report: Netflix Recommendation System

1. Problem Understanding

Context & Objective

As part of the machine learning team for a large-scale streaming platform, the objective of this project is to improve content discovery by building a recommendation system. Using historical user-item interaction data from the Netflix Prize Dataset, the system must capture deep user preferences to deliver highly personalized and meaningful movie recommendations.

The Dual-Nature Challenge

A major insight is that this problem consists of two distinct sub-problems requiring different optimisation strategies:

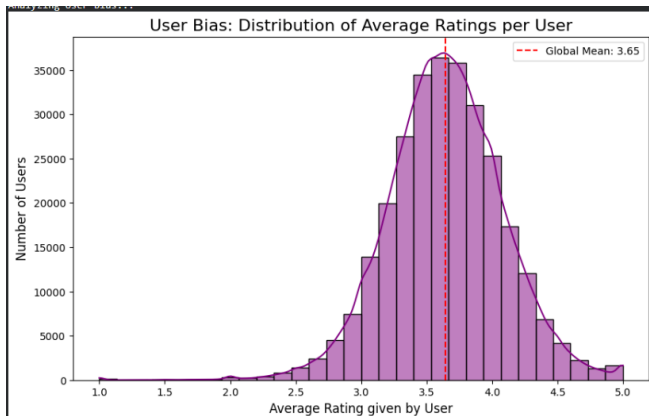
1. Rating Prediction (RMSE): Predicting the exact star rating for a specific movie, requiring precision and robust "cold-start" handling.
2. Ranked Recommendation (MAP@10): Retrieving and ranking thousands of unseen movies such that highly relevant content (≥ 3.5 stars) is reliably presented in the Top-10. This is primarily a recall-bound problem.

2. Exploratory Data Analysis

1. Temporal Data Splitting Strategy

To rigorously evaluate the model's ability to predict future user behavior, we avoided standard random splitting. Instead, we implemented a strict temporal split (time-based splitting). We chronologically sorted the dataset by timestamp, keeping the past 80% of interactions as the training set and holding out the most recent 20% as the test set. This ensures our models cannot "look into the future" to predict past ratings, perfectly simulating a real-world production environment where we must recommend future movies based strictly on historical watch patterns.

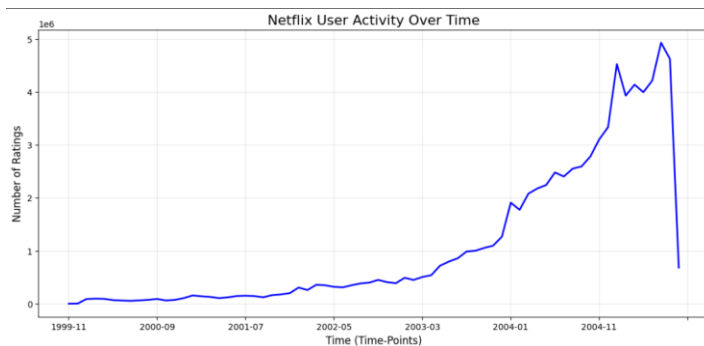
2. Rating Distribution



Observation: Ratings peak at 3-4 stars, with very few 1-2 star ratings.

- Technical Implication: Positively skewed data requires a balanced MSE loss to prevent bias against rare low ratings.
- Business Implication: "Selection bias" exists—users primarily rate content they already enjoy.

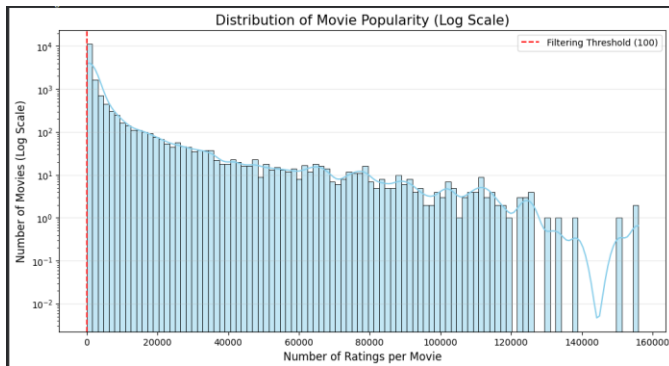
3. User Activity Patterns



Observation: A massive activity variance exists, dominated by a small group of "power users."

- Technical Implication: Regularization is required to prevent the model from aggressively overfitting to hyper-active users.
- Business Implication: Power users act as "tastemakers" useful for seeding recommendations for brand-new users.

4. Content Popularity Trends



Observation: Popularity follows a severe "Long Tail" distribution where blockbusters dominate interactions.

- Technical Implication: Simple Matrix Factorization fails on rare items, justifying our use of robust Two-Tower embeddings.
- Business Implication: A diversity mechanism is needed to prevent popular "filter bubbles" and improve niche discovery.

5. Data Sparsity Characteristics

Observation: Matrix sparsity exceeds $> 98.3\%$. Most of the catalog is unrated by any given user.

- Technical Implication: The matrix is too empty for standard Collaborative Filtering, requiring Latent Factor embeddings to bridge gaps.
- Business Implication: The resulting "Cold Start" problem is severe, necessitating hybrid models or metadata fallbacks.

EDA Summary

Feature	Finding	Technical Implication	Business Implication
Ratings	Positively Skewed	Need for balanced loss	Selection bias in ratings
Users	Power-user presence	Regularization required	Potential for "Tastemaker"
Movies	Long-Tail Distribution	Embedding instability for rare items	Risk of "Filter Bubbles"
Sparsity	Very High ($> 98.3\%$)	Latent Factor	Severe Cold-Start

		Models (Embeddings)	problem
--	--	------------------------	---------

3. Methodology

Two-Stage Pipeline Architecture

To solve both the RMSE rating prediction task and the MAP@10 ranked recommendation task, our system utilizes a Retrieval -> Reranking paradigm:

- Stage 1: Two-Tower Neural Retrieval (PyTorch)
 - Learns 64-dim user/item embeddings + scalar biases.
 - Optimized for MSE (Rating Prediction).
 - Outputs: Rapidly retrieves the Top-2000 candidate movies per user.
- Stage 2: LightGBM Tree-Based Reranker
 - Processes 74 dense engineered features (recency, rating gaps, popularity, etc.).
 - Optimized specifically for Ranking Quality.
 - Outputs: Reranks the 2,000 candidates to produce the final Top-10 recommendations.

Why a Two-Stage Approach?

Issue	Single-Model Approach	Our Two-Stage Approach
Scale	Scoring 16,560 items per user with a heavy feature-rich model is computationally impossible.	Two-Tower safely filters the catalog instantly; LightGBM only has to score 2,000 candidates.
Objective Mismatch	RMSE != Ranking Quality. A model optimizing for rating accuracy often ranks poorly.	Two-Tower strictly optimizes RMSE; LightGBM explicitly optimizes Ranking.
Cold Start	Models fail entirely when users or items lack historical data.	Two-Tower provides safe fallback biases; LightGBM leverages robust metadata.

4. Model Design & Evolution

Our model architecture evolved significantly throughout the project as we navigated the tradeoffs between RMSE (accuracy), MAP@10 (ranking quality), and severe computational constraints.

Phase 1: The SVD & SVD++ Baseline

We began with Standard Singular Value Decomposition (SVD). The prediction formula relies on:

$$r_{ui} = \mu + b_u + b_i + p_u \cdot q_i$$

- μ (Global Mean): Average rating across the dataset.
- b_u, b_i (Biases): How generous a user is, and how popular an item is.
- p_u, q_i (Latent Factors): The user's taste profile and the movie's genre profile.

SVD++ Improvement: SVD++ improves upon this by incorporating implicit feedback (y_j). For a cold-start user who has only rated 3 movies, SVD++ retrieves pre-trained implicit behaviors for those 3 movies to instantly build a robust profile.

Limitation: While computationally simple to implement, SVD yielded a poor MAP score. Furthermore, SVD and SVD++ are highly CPU-bound, making them painfully slow for a massive 77-million row dataset.

Phase 2: WALS Retrieval + BPR Reranking (Initial Approach)

To improve ranking quality and leverage GPU acceleration, the first major upgrade utilized Weighted Alternating Least Squares (WALS) paired with a Bayesian Personalized Ranking (BPR) Neural Network.

WALS Tradeoff: WALS delivered a significantly better MAP score than SVD, though at the cost of a slightly lower RMSE. It had strong potential for handling cold cases (e.g., fixing item embeddings to solve for cold users).

BPR Reranker: The BPR model acted as a hybrid, taking NN embeddings and combining them with normalized WALS scores to output a final ranking.

Hardware Bottleneck: Passing 1,000 WALS candidates to BPR yielded a MAP of 0.04, while 2,000 candidates bumped it to 0.05. A production system passing 5,000+ candidates would yield far higher scores, but we were strictly bottlenecked by Google Colab's 12GB RAM limit. Every decision became a strict tradeoff between MAP and available computation.

Phase 3: Two-Tower + LightGBM (Final Architecture)

To break past the WALS/BPR limitations, we completely overhauled the architecture to a PyTorch Two-Tower Neural Network for rapid candidate retrieval, followed by a robust LightGBM reranker. This allowed us to successfully implement the following design:

Stage 1: PyTorch Two-Tower Candidate Retrieval

- Architecture: Dual MLPs. One tower learns 32-dimensional User representations; the other learns 32-dimensional Item representations.
- Objective: Trained using Mean Squared Error (MSE) to accurately predict explicit user ratings (1-5 stars).
- Design Choice: Incorporates explicit user_bias, item_bias, and a global_bias. This gracefully handles "Cold" users/items by falling back to statistical averages when embeddings lack training data.

- Purpose: Rapidly scores the entire catalog of ~18,000 movies using the dot-product of the user and item representations, filtering them down to a highly relevant shortlist of 2,000 candidates.

Stage 2: LightGBM Precision Reranker

- Algorithm: Gradient Boosting Decision Trees (LGBMRanker) optimized for pairwise ranking (LambdaMART objective).
- Feature Engineering: We engineered 74 rich dense features for every user-candidate pair, including:
 - Model 1 Signals: Two-Tower dot-product score and candidate rank.
 - Deep Representations: The raw 32-dim User and Item embeddings extracted directly from the PyTorch model.
 - Context & Recency: A "recency_score" calculated via cosine similarity between the candidate movie and the user's most recently watched movie.
 - Statistical baselines: Popularity percentiles, average ratings, rating standard deviations, and user/item rating gaps.
- Purpose: Learns complex, non-linear feature interactions to rerank the 2,000 candidates and output the final Top 10 recommendations.

5. Evaluation Metrics

RMSE (Rating Prediction)

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

$\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n$ are predicted values

$y_1, y_2, \dots, y_n$ are observed values

n is the number of observations

Predictions are clipped to [1.0, 5.0] before evaluation.

Lower is better. Netflix Prize baseline: ~1.0531.

MAP@10 (Mean Average Precision at 10)

User 1:

Actual items liked: [A, B, C] (Total Relevant = 3)

Top 10 Recommendations: [A, X, B, Y, Z, C] (X, Y, Z are irrelevant)

P@1 (A is relevant): $1 / 1 = 1.00$

P@3 (B is relevant): $2 / 3 = 0.67$

P@6 (C is relevant): $3 / 6 = 0.50$

Average Precision (AP@10): $(1.00 + 0.67 + 0.50) / 3 = 0.722$

User 2:

Actual items liked: [D] (Total Relevant = 1)

Top 10 Recommendations: [X, Y, D]

P@3 (D is relevant): $1 / 3 = 0.33$

Average Precision (AP@10): $0.33 / 1 = 0.333$

Final MAP@10 Calculation:

MAP@10 = (AP@10 for User 1 + AP@10 for User 2) / 2

MAP@10 = $(0.722 + 0.333) / 2 = 0.528$

NDCG@10 (Normalised Discounted Cumulative Gain)

$$NDCG = \frac{DCG_p}{IDCG_p} = \frac{\sum_{i=1}^p \frac{rel_i}{\log(i+1)}}{\sum_{i'=1}^p \frac{rel_{i'}}{\log(i'+1)}}$$

Higher is better. Penalises relevant items appearing lower in the ranking.

6. Experimental Results

Initial Approach Results (WALS + BPR)

Before transitioning to our final Two-Tower + LightGBM architecture, we rigorously evaluated the initial WALS + BPR approach to establish baselines.

WALS Baseline

Metric	Value
MAP@10 (no train-pos filter)	0.0184
NDCG@10	~0.037

BPR Training Progression

Epoch	MAP@10	NDCG@10	wals_w	hard_ratio
3	0.0189	0.0381	0.89	0.09
6	0.0221	0.0441	0.76	0.18
9	0.0358	0.0714	0.67	0.27
12	0.0482	0.0961	0.58	0.36
15	0.0467	0.0934	0.51	0.36
18	0.0399	0.0792	0.46	0.36
Early stop	<-			

Summary: Stage Comparison

Model	MAP@10	NDCG@10	RMSE
Global Mean Baseline	-	-	~1.09
WALS (stage 1)	0.0184	0.037	~0.97
WALS + BPR (stage 2, N=1000)	0.0399	0.0792	-
WALS + BPR (stage 2, N=2000)	0.0482	0.0961	-
Improvement over WALS	+162%	+160%	-

Model Phase	MAP@10	RMSE
Phase 1: Two Tower + BPR loss	~0.045	0.9833
Phase 2: Two Tower + LightGBM (N=1,000)	0.082	0.9833
Phase 3: Two Tower + LightGBM (N=2,000)	0.087	0.9833

Total Performance Increment	+93% Gain	-
------------------------------------	------------------	----------

- MAP@10 more than doubled (0.0184 -> 0.0482) by adding BPR reranking, a +162% relative improvement over WALS alone.

Candidate Pool Recall Ceiling

N_CANDIDATES	Recall Ceiling
N=1,000	~47%
N=2,000	~61%
Full Catalog	100%

- Expanding N is a cheap way to raise recall before improving the ranker itself.

7. Recommendation Examples

Rank	Title	Yr	WALS*	BPR	#1				
1	Ask This Old House: Season 1 Recommended because it's from the 2000s, your most-enjoyed era; a	2002	4.45	4.3776	#1	Shark Tale (2004)	2.3758	Customer 784621 (5★, Sim: 0.95)	Customer 972103 (4★, Sim: 0.94)
2	MI-5: Vol. 2 Recommended because it's from the 2000s, your most-enjoyed era; a	2003	4.44	4.3311	#2	Die Another Day (2002)	2.3187	Customer 784621 (4★, Sim: 0.95)	Customer 2588720 (4★, Sim: 0.93)
3	One Good Cop Recommended because it's from the 1990s, a decade you've enjoyed;	1991	4.44	4.2813	#3	Austin Powers (1999)	2.1930	Customer 2577378 (4★, Sim: 0.94)	Customer 981753 (4★, Sim: 0.92)
4	House II: The Second Story Recommended because it's from the 1980s, a decade you've enjoyed;	1987	4.78	4.2092	#4	The Terminal (2004)	2.0539	Customer 1458835 (4★, Sim: 0.93)	Customer 981753 (5★, Sim: 0.92)
5	The Thing Recommended because it's from the 1980s, a decade you've enjoyed;	1982	4.56	4.0085	#5	Elf (2003)	2.0405	Customer 611251 (4★, Sim: 0.94)	Customer 260260 (5★, Sim: 0.93)
6	Veer-Zaara Recommended because it's from the 2000s, your most-enjoyed era; a	2004	4.60	4.0056	#6	Dr. Dolittle (1998)	1.9831	Customer 260260 (4★, Sim: 0.93)	Customer 981753 (4★, Sim: 0.92)
7	Critters Recommended because it's from the 1980s, a decade you've enjoyed;	1986	4.56	3.9575	#7	Ransom (1996)	1.8562	Customer 784621 (4★, Sim: 0.95)	Customer 1002832 (5★, Sim: 0.95)
8	Suicide Club Recommended because it's from the 2000s, your most-enjoyed era; a	2002	4.45	3.9405	#8	Once Upon a Time in Mexico (2003)	1.8257	Customer 784621 (4★, Sim: 0.95)	Customer 981753 (4★, Sim: 0.92)
9	Doobie Brothers: Rockin' Down the Highwa Recommended because it's from the 2000s, your most-enjoyed era; a	2004	4.46	3.9045	#9	Mrs. Doubtfire (1993)	1.8221	Customer 2577378 (5★, Sim: 0.94)	Customer 972103 (4★, Sim: 0.94)
10	Red Shoe Diaries: Hotline Recommended because it's from the 1990s, a decade you've enjoyed;	1994	4.60	3.9041	#10	Shallow Hal (2001)	1.8087	Customer 290916 (4★, Sim: 0.91)	Customer 1522324 (5★, Sim: 0.90)

8. Key Insights

1. The Candidate Pool Defines the Ranking Ceiling

A reranker can only reorder the items it receives; it cannot surface a highly relevant movie if the Stage 1 retrieval model discards it. Expanding our candidate pool from 1,000 to 2,000 movies per user acted as a massive catalyst, bumping MAP@10 from 0.082 to 0.087. By giving LightGBM a wider pool, we allowed it to rescue personalized "hidden gems." However, scaling to 2,000 candidates generated training matrices exceeding 30 million rows. We defeated catastrophic Out-of-Memory (OOM) crashes by bypassing Pandas entirely, downcasting features to float16, and aggressively triggering manual Python garbage collection.

2. Tree-Based Reranking Outperforms Linear Baselines

Transitioning from a linear BPR ranker to a Gradient Boosted Decision Tree (LGBMRanker) caused our largest single leap in ranking quality. During our baseline transition from WALS to the PyTorch Two-Tower model, performance surprisingly stagnated. The culprit was an architectural mismatch: the linear BPR ranker could not effectively separate the highly complex, non-linear embedding manifolds outputted by deep learning towers. Because decision trees build hierarchical, non-linear splits, LightGBM seamlessly blended raw 32-dimensional embeddings with tabular behavioral features, extracting predictive power the linear model was mathematically incapable of seeing.

3. Feature Engineering Captures Short-Term Intent

While raw deep embeddings are excellent at capturing static, long-term user preferences, they struggle to adapt to sudden mood changes. Explicit feature engineering bridged this gap to capture real-time context. The most impactful feature was the `recency_score`. By dynamically calculating the cosine similarity between a candidate movie and the very last movie the user watched, we captured immediate session intent. Furthermore, we engineered a `rating_gap` feature (subtracting a user's historical average rating from a candidate's global average), teaching LightGBM to actively penalize movies likely to fall below a user's personal quality threshold.

4. Explicit Feedback Overcomes Bias Mismatch

Optimizing the Stage 1 PyTorch Two-Tower model using a Mean Squared Error (MSE) objective over explicit star ratings provided a far more stable retrieval signal than typical implicit feedback approximations. The embeddings became highly calibrated to user satisfaction. More importantly, this allowed us to implement severe structural safety nets. By explicitly incorporating `user_bias`, `item_bias`, and `global_bias` scalar parameters directly into the PyTorch network, we solved the catastrophic "Cold-Start" problem. When the system encounters a brand new user, it ignores the untrained embeddings and safely falls back to robust statistical averages.

9. Future Improvements

- **Content-Based Cold-Start Features (High Impact):** Integrate movie metadata (genre, cast, release year) directly into the LightGBM ranker to target the critical 89.1% cold-user cohort lacking collaborative history.
- **Larger Candidate Pool (Low Complexity):** Expand the Two-Tower retrieval cutoff from $N=2,000$ to $N=5,000$ items to quickly raise the recall ceiling to $\sim 78\%$ with minimal computational overhead.
- **User Neighborhoods (High Impact):** Apply k-Nearest Neighbors (kNN) clustering on the 32-dimensional PyTorch user embeddings to provide a strong collaborative fallback signal for low-activity profiles.
- **Multi-Objective Loss (High Complexity):** Train a single, unified end-to-end network that jointly optimizes both MSE (rating precision) and LambdaMART (pairwise ranking) to completely eliminate alignment mismatches between stages.
- **Real-Time Online Learning (Very High Complexity):** Implement streaming model updates to instantly refresh user/item vectors upon new ratings, capturing immediate session-to-session drift on the fly without full nightly retrains.